

Holder–Verifier Protocol Specification

Aadhaar mDoc + Zero-Knowledge system — Aadhaar Holder (`com.mdochoolder.app`) and Aadhaar Verifier (`com.example.mdocverifier`).

This document specifies every message exchanged between the Holder and the Verifier, on every transport, for both presentation modes. For each request and response it defines the structure and the meaning of every field, cites the corresponding clause of **OpenID for Verifiable Presentations 1.0** (the published Final specification, referred to as *OpenID4VP 1.0*), and documents one captured presentation end to end with the actual bytes and the code that runs on each side. The complete captured artifacts are in [docs/samples/](#).

The captured session. Every decoded request and response in this document is from a real online session `9d7895aa11e55fff042cc1723f5bd78b` , captured live, in which the Verifier requested four age attributes (`age_above18` , `age_above60` , `age_above50` , `age_above75`) and the Holder returned a proof disclosing their values. The request is a **signed** JWS with the Verifier certificate in `x5c` ; the response is **encrypted** as a `direct_post.jwt` JWE. Sections 7 and 8 add separately captured **Bluetooth** and **NFC** sessions from the same build. Raw artifacts: [samples/request.jwt](#), [samples/request.decoded.json](#), the full encrypted response [samples/response.jwe.txt](#), the complete decrypted vp_token [samples/response.vp_token.txt](#) and its full decode [samples/response.decoded.txt](#), and the session transcript [samples/session-transcript.hex](#).

Security profile. This build implements OpenID4VP 1.0's high-assurance options on every zero-knowledge transport: the Authorization Request is **signed** (ES256 JWS, verifier certificate in the `x5c` header, `x509_san_dns` client identifier), the Holder **verifies the signature and pins the certificate** to the one bundled in the app (a two-layer check), and — online — the response is **encrypted** to the Verifier's ephemeral key (`direct_post.jwt` , JWE ECDH-ES / A256GCM). Section 15 is a clause-by-clause conformance map to OpenID4VP 1.0.

Contents

- [1. Presentation modes](#)
- [2. Terminology](#)
- [3. Data model](#)
- [4. What is disclosed, and what is not](#)
- [5. The zero-knowledge engine](#)
- [6. Transport A — Online \(relay\)](#)
- [7. Transport B — Bluetooth Low Energy](#)
- [8. Transport C — NFC engagement to Bluetooth](#)
- [9. Selective disclosure \(ISO 18013-5\)](#)
- [10. Session transcript](#)
- [11. Error conditions](#)
- [12. Field reference](#)
- [13. Worked example — the captured presentation, step by step](#)
- [14. Requesting four attributes, disclosing fewer](#)
- [15. OpenID4VP 1.0 conformance map](#)

1. Presentation modes

The Holder presents its Aadhaar mdoc in one of two modes.

Zero-knowledge proof. The Holder discloses the requested attribute values (for example, `age_above18 = "Yes"`) together with a proof that those values are genuinely signed by the issuer (UIDAI). The issuer's signature over the MSO is **not transmitted** — its validity is proven inside the zero-knowledge circuit — and no attribute other than those requested is revealed. The verifier learns the requested values and is cryptographically assured of their authenticity and of holder binding.

Selective disclosure (ISO 18013-5). The Holder discloses the requested attribute values together with the issuer-signed Mobile Security Object (MSO). The verifier checks each value against its digest in the MSO and verifies the MSO signature directly. The actual values and the issuer signature are transmitted.

Property	Zero-knowledge proof	Selective disclosure
Requested attribute values	Disclosed	Disclosed
Non-requested attributes	Not revealed	Not revealed
Issuer signature over the MSO	Proven in zero-knowledge; not transmitted	Transmitted
Issuer certificate (public key)	Transmitted (the Document Signer certificate)	Transmitted
Holder binding	Device signature verified inside the circuit	Device MAC / signature
Request authentication	Signed request (ES256 JWS + x5c) + certificate pinning	ISO reader authentication (optional)
Response confidentiality	Encrypted online (direct_post.jwt , JWE); AES-256-GCM on Bluetooth	AES-256-GCM (ISO session encryption)
Protocol	OpenID4VP 1.0 (format = mso_mdoc_zk)	ISO 18013-5 DeviceRequest / DeviceResponse
Transports	Online (relay), Bluetooth LE, NFC to Bluetooth	Bluetooth LE (QR or NFC engagement)

Sections 6–8 specify the zero-knowledge transports; section 9 specifies selective disclosure.

1.1 Transports: online and offline

A presentation is delivered over one of three transports. They differ only in how the request and the proof travel between the two devices; the request object and the proof themselves are identical across all three (sections 6–8).

- **Online (relay).** The two devices do not communicate directly. A small store-and-forward server on the internet — the relay (section 6.5) — holds the two messages: the Verifier uploads the signed request, the Holder downloads and verifies it, generates the proof, encrypts it, and uploads it, and the Verifier downloads and decrypts it. The devices may be on different networks and any distance apart, provided both have internet access. The engagement (a QR code or a same-device deep link) carries an HTTPS URL identifying the relay session.
- **Offline — Bluetooth Low Energy.** The Holder connects directly to the Verifier over Bluetooth and transfers the proof device-to-device, with no server and no internet. The devices must be within Bluetooth range (a few metres). The engagement (a QR code) carries the Verifier's Bluetooth identity and its X25519 public key, which the Holder uses to establish the encrypted channel; the signed request travels over that channel.
- **Offline — NFC to Bluetooth.** Identical to the Bluetooth transport, except the engagement is delivered by tapping the two devices together (NFC) instead of scanning a QR code. NFC carries only the small engagement; the signed request and the proof are transferred over Bluetooth.

The choice of transport does not change the request, the proof, or the verification — only the channel and the response confidentiality mechanism.

2. Terminology

Term	Definition
mdoc	The credential: an ISO/IEC 18013-5 mobile document in CBOR, containing <code>issuerSigned</code> items and the MSO
Namespace / element	An attribute is addressed as <code>[namespace, element]</code> , e.g. <code>["in.gov.uidai.aadhaar.1", "age_above18"]</code>
IssuerSignedItem	One attribute record: <code>digestID</code> , a random salt, <code>elementIdentifier</code> , <code>elementValue</code>
MSO	Mobile Security Object; an issuer-signed structure committing to a digest of every attribute and to the device public key
Document Signer (DS)	The issuer key/certificate that signs the MSO; its certificate is carried in <code>msoX5chain</code> (<code>CN = UIDAI DS - in.gov.uidai.aadhaar.1</code>)
IACA	Issuing Authority Certification Authority — the root that issues DS certificates (<code>CN = UIDAI IACA Root CA</code>); the verifier's issuer trust anchor, held out-of-band and never transmitted
Device key	An EC P-256 key pair (<code>mdoc_device_key</code>) in the Android Keystore; its public half is in the MSO
Holder binding	The property that a presentation is signed by the device key in the MSO, proving the credential is on the device it was issued to
OpenID4VP	The presentation request/response protocol (this build targets OpenID4VP 1.0 Final)
JAR	JWT-Secured Authorization Request (RFC 9101) — the request delivered as a signed JWT via <code>request_uri</code>
Signed request (<code>x5c</code>)	The Authorization Request as an ES256 JWS whose header <code>x5c</code> carries the Verifier's X.509 certificate
<code>x509_san_dns</code>	The client identifier prefix (OpenID4VP 1.0 section 5.9.3) in which the <code>client_id</code> is a DNS name present in the certificate's SubjectAltName
Certificate pinning	The Holder additionally requires the request's <code>x5c</code> leaf to be byte-identical to the verifier certificate bundled in the app
<code>direct_post.jwt</code>	The encrypted response mode (OpenID4VP 1.0 section 8.3.1): the response is returned as a JWE
JWE (ECDH-ES / A256GCM)	The encrypted response: ephemeral-static ECDH key agreement (<code>alg=ECDH-ES</code>) with AES-256-GCM content encryption (<code>enc=A256GCM</code>)
JWK thumbprint	RFC 7638 SHA-256 thumbprint of the Verifier's encryption public key; folded into the session transcript when the response is encrypted
DCQL	Digital Credentials Query Language (OpenID4VP 1.0 section 6) — the query inside the request
vp_token	The response object carrying the DeviceResponse (OpenID4VP 1.0 section 8.1)
DeviceResponse	The ISO 18013-5 response container; its <code>zkDocuments</code> entry carries the zero-knowledge proof
ZkDocument	The zero-knowledge document: the proof plus the disclosed values and the issuer certificate
Circuit	A longfellow circuit compiled for a fixed attribute count, identified by a deterministic id (<code>zkSystemId</code> in CBOR; the Kotlin API exposes it as <code>zkSystemSpecId</code>)
Session transcript	A value both parties compute identically; the proof is bound to it
Relay	A store-and-forward mailbox used only by the online transport

3. Data model

```
mdoc (stored credential, CBOR)
├─ docType = "in.gov.uidai.aadhaar.1"
├─ issuerSigned
│   └─ nameSpaces
│       └─ "in.gov.uidai.aadhaar.1": [ IssuerSignedItem, IssuerSignedItem, ... ]
│           └─ IssuerSignedItem = { digestID, random (salt), elementIdentifier, elementValue }
├─ issuerAuth = COSE_Sign1( MSO )          ← the issuer's (UIDAI) signature
      MSO commits to: a digest of every element, and the device public key
```

The response the Holder returns is an ISO 18013-5 **DeviceResponse**. In zero-knowledge mode it carries a `zkDocuments` entry. This is the real decoded structure from the captured session:

```

DeviceResponse
├── version = "1.0"
├── zkDocuments = [ ZkDocument ]
│   └── ZkDocument
│       ├── proof : 364228 bytes (native longfellow output)
│       └── documentData : 24( bstr( { ← CBOR tag 24 (encoded-CBOR) wrapping a map
│           ├── zkSystemId : "longfellow-libzk-v1_7_4_4415_4096_5aebda...e460" ← the circuit id
│           ├── docType : "in.gov.uidai.aadhaar.1"
│           ├── timestamp : 0("2026-07-22T12:51:39Z")
│           └── issuerSigned: { "in.gov.uidai.aadhaar.1":
│               [ {elementIdentifier:"age_above18", elementValue:"Yes"},
│                 {elementIdentifier:"age_above60", elementValue:"No"},
│                 {elementIdentifier:"age_above50", elementValue:"No"},
│                 {elementIdentifier:"age_above75", elementValue:"No"} ] }
│           ├── deviceSigned: {} ← empty; device binding is inside the proof
│           └── msoX5chain : <662 bytes DER issuer certificate>
│       } ) )
└── status = 0

```

(The Kotlin API exposes `documentData.issuerSigned` as a `Map<namespace, Map<element, value>>` ; on the wire it is the array of `{elementIdentifier, elementValue}` maps shown above.)

4. What is disclosed, and what is not

In zero-knowledge mode the message on the wire (after the Verifier decrypts the response) contains:

- **Disclosed** — the requested attribute values (`documentData.issuerSigned`), the circuit id (`zkSystemId`), the presentation timestamp, and the Document Signer certificate (`msoX5chain`).
- **Not transmitted** — the issuer's signature over the MSO (proven inside the circuit), the digests and values of every non-requested attribute, and the device signature (also proven inside the circuit; `deviceSigned` in the document is empty).

Proving "age over 18" without revealing the date of birth is achieved by disclosing the attribute `age_above18` (value `"Yes"`) while the `dob` attribute is simply not among the requested claims and never leaves the device.

5. The zero-knowledge engine

Engine: Google **longfellow-zk** via OpenWallet Foundation **Multipaz** (`LongfellowZkSystem`). The Holder is the prover (`ZkPresentationManager`); the Verifier verifies (`ZkVerifier` → `LongfellowZkSystem.verifyProof`).

5.1 Prover — inputs and output

Inputs to `generateZkDocument` :

Input	Description	Source
<code>mdocBytes</code>	The stored credential (CBOR with <code>issuerSigned</code> and MSO)	<code>filesDir/aadhaar_mdoc_b64.txt</code>
<code>docType</code>	<code>"in.gov.uidai.aadhaar.1"</code>	The request / the credential
<code>selected</code>	The <code>[namespace, element]</code> pairs to disclose	The request's DCQL claims
<code>encodedSessionTranscript</code>	The session transcript bytes (section 10)	Computed from the request or the Bluetooth keys
<code>devicePrivateKey</code>	The <code>mdoc_device_key</code> private key	Android Keystore
<code>timestamp</code>	Presentation time	<code>Clock.System.now()</code>

Steps: (1) reject attributes exceeding the size limits; (2) select the circuit by attribute count; (3) filter the `IssuerSigned` items to the selected attributes; (4) build an ES256 device signature over `DeviceAuthentication` for holder binding; (5) call `generateProof` . Output: a `ZkDocument` , wrapped by `buildZkDeviceResponse` into a `DeviceResponse{ zkDocuments: [ZkDocument] }` .

5.2 Verifier — inputs and output

Inputs to `verifyDeviceResponse` : the `DeviceResponse` bytes (after decrypting the `direct_post.jwt` response online, or the Bluetooth channel offline) and the `transcript` bytes the Verifier computed independently for this session.

Verification consists of two parts. First, issuer trust: the Verifier validates the Document Signer certificate carried in `msoX5chain` against its bundled UIDAI IACA trust anchor (section 5.4); if the certificate does not chain to the IACA, the presentation is rejected before the proof is checked. Second, proof verification: `LongfellowZkSystem.verifyProof(zkDocument, spec, transcript)` verifies, in zero-knowledge: (a) a valid issuer signature exists over an MSO committing to the disclosed attributes, without the signature being transmitted; (b) the disclosed attribute values are consistent with the digests the MSO commits to; (c) a device signature over this exact transcript is valid against the device key in the MSO. The call throws on any failure. Output — `ZkVerifier.Result` :

Field	Meaning
<code>success</code>	<code>true</code> if and only if <code>verifyProof</code> did not throw
<code>docType</code>	The credential document type
<code>disclosedAttributes</code>	<code>{ element: value }</code> — the disclosed values read from the proof
<code>zkSystemSpecId</code>	The circuit id used (the <code>zkSystemId</code> from the document)
<code>issuerCertSubject</code>	The issuer certificate subject
<code>holderBound</code>	<code>true</code> on success (device signature verified in the circuit)

5.3 Circuits, size limits, and circuit selection

A longfellow circuit is compiled for a fixed number of attributes. The Holder selects a circuit from its bundled set by matching the number of attributes it is disclosing; the selected circuit's id is written into the proof as `zkSystemId`. **The `zkSystemId` in the proof is authoritative**: the Verifier resolves the circuit by looking that id up in its own bundled set (`systemSpecs.first { it.id == specId }`) and verifies against it.

The request also advertises a circuit under `meta.zk_system_type` for reference; the Holder is not required to use it. In the captured session the request advertised `longfellow-libzk-v1_6_4_...` (version 6) while the Holder proved with `longfellow-libzk-v1_7_4_...` (version 7); verification succeeded because the Verifier resolved the circuit from the proof's `zkSystemId` and both applications bundle that circuit. Consequently the number of attributes in the proof is decided by the Holder, which is the basis for section 14.

An attribute is provable only if its CBOR value is at most 64 bytes and its `IssuerSignedItem` is at most 119 bytes; larger attributes (for example a photograph or a long address) are disabled in the consent interface and excluded from the proof. The proof is approximately 360 KB (the captured proof is 364,228 bytes). Because of this size, the Bluetooth transport chunks it and the online transport uses a mailbox.

5.4 Issuer trust: the IACA and the Document Signer certificate

The authenticity of the credential rests on a two-level certificate hierarchy.

- The **MSO** is signed by a **Document Signer (DS)** key. The DS certificate is transmitted with the response as `msoX5chain` — a single X.509 certificate carrying the DS public key and identity (`CN = UIDAI DS - in.gov.uidai.aadhaar.1` ; 662 bytes in the captured session).
- The DS certificate is issued by the **IACA** root (`CN = UIDAI IACA Root CA`). The IACA is the issuer trust anchor. It is not transmitted; the Verifier holds it out-of-band, bundled at `res/raw/uidai_iaca_cert`.

The Verifier establishes trust in the following order (`ZkVerifier.validateIssuer`), before it checks the proof:

1. Read the DS certificate from `msoX5chain`.
2. Verify the DS certificate's signature against the IACA public key held by the Verifier, confirming the DS certificate was issued by UIDAI (`dsCert.verify(iacaCert.publicKey)`).
3. Confirm the DS certificate is within its validity period and names the IACA as its issuer.
4. Verify the proof, which establishes that the MSO was signed by the DS key.

If the DS certificate does not chain to the IACA, the presentation is rejected ("Issuer certificate not trusted: ..."). The result is a complete chain of trust: the IACA, pre-provisioned in the Verifier, authenticates the DS certificate; the DS key authenticates the MSO; and the MSO commits to the disclosed attributes and the device key. On success the Verifier's result screen displays "Issued by UIDAI".

Two distinct trust anchors. The **IACA** (above) is how the Verifier trusts the *issuer* of the credential. Section 5.5 covers the reciprocal direction — how the *Holder* trusts the *Verifier* that sends the request. They are independent: one authenticates the document, the other authenticates the requester.

5.5 Verifier authentication: signed request and certificate pinning

On every zero-knowledge transport the Authorization Request is authenticated so the Holder releases data only to a Verifier it trusts. This is a **two-layer check** performed by the Holder (`VerifierRequestAuth.verifyAndExtractClaims` , shared by all transports):

1. **Signature (OpenID4VP 1.0 section 5, JAR / RFC 9101).** The request is an ES256 JWS with the Verifier's X.509 certificate in the `x5c` header. The Holder parses the JWS, takes the leaf certificate's public key, and requires the signature to verify against it. This proves the request was produced by the holder of that certificate's private key and was not altered in transit.
2. **Pinning.** The Holder additionally requires that leaf certificate to be **byte-identical** to the verifier certificate bundled in the app (`res/raw/verifier_cert`). This proves it is *the specific Verifier this app trusts*, not merely any party able to present a well-formed certificate.

```
// VerifierRequestAuth.verifyAndExtractClaims (common module)
val signed = SignedJWT.parse(requestJwt)
val leaf = x509FromBase64(signed.header.x509CertChain[0]) // the x5c leaf
require(signed.verify(ECDSAVerifier(leaf.publicKey as ECPublicKey))) // (1) signature
require(leaf.encoded.contentEquals(trustedCert.encoded)) // (2) pin
return signed.payload.toString() // the request claims JSON
```

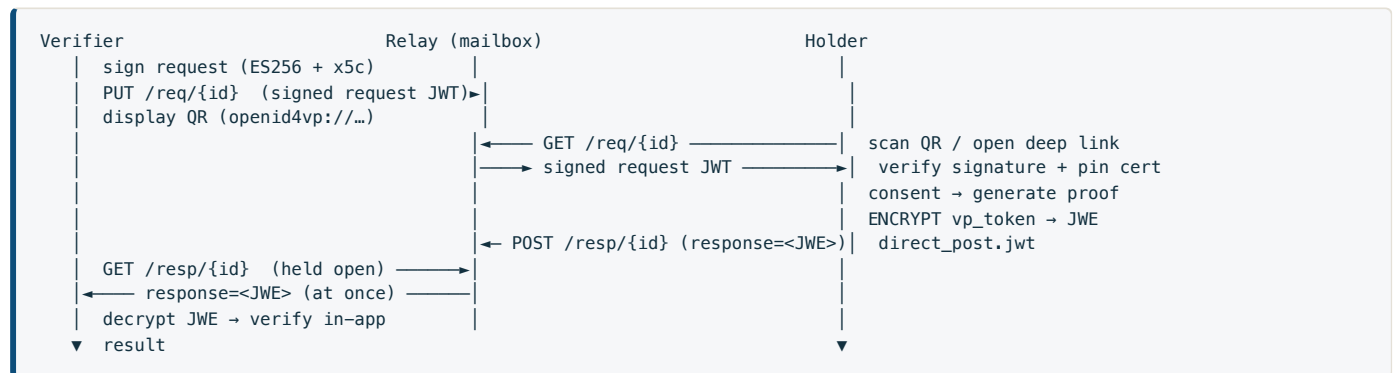
Only if both checks pass are the request claims used. If a Verifier certificate is pinned and the request is unsigned, it is refused outright (no silent fallback) — identical rule on online, Bluetooth, and NFC. On success the Holder logs `Request signature verified and verifier certificate pinned OK`.

The `client_id` is `x509_san_dns:aadhaar-verifier`; the part after the `x509_san_dns:` client identifier prefix (OpenID4VP 1.0 section 5.9.3) is `aadhaar-verifier`, the DNS name in the certificate's SubjectAltName. Relationship to the trust models: the standard `x509_san_dns` scheme expects the certificate to chain to a trust list of authorized verifiers; this build **pins one bundled certificate** instead — a stricter, closed-deployment choice (a trust list of size one) appropriate for a single-Verifier Aadhaar pilot. A production deployment would replace the pin with a managed, revocable verifier trust list.

6. Transport A — Online (relay)

Verifier: `0idVpVerifier.kt`, `VerifierCrypto.kt`. Holder: `OpenId4VpZkClient.kt`, `VerifierRequestAuth.kt`, `OpenId4VpActivity.kt`. Relay: `worker.js`, base `https://oid4vp-relay.praveen-mdoc.workers.dev`.

6.1 Sequence



6.2 The QR / deep link (engagement)

The Verifier displays this string as a QR code and uses the identical string as a same-device deep link (the two are protocol-identical; same-device is an `ACTION_VIEW` intent, cross-device is the same URI scanned):

```
openid4vp:///client_id=<url-encoded client_id>&request_uri=<url-encoded request_uri>
```

`openid4vp:///...` is the OpenID4VP Authorization Request (OpenID4VP 1.0 section 5); `request_uri` is an HTTPS URL the wallet performs a `GET` on to retrieve the request object (Request URI method `get`, JAR / RFC 9101; OpenID4VP 1.0 section 5.10). Captured engagement (session `9d7895aa...`):

```
openid4vp:///client_id=x509_san_dns%3Aaadhaar-verifier&request_uri=https%3A%2F%2Foid4vp-relay.praveen-mdoc.workers.dev%2Freq%2F9d7895aa11e55fff042cc1723f5bd78b
```

The `client_id` carries the OpenID4VP 1.0 client identifier prefix `x509_san_dns:` in front of the identity `aadhaar-verifier` (section 5.9.3); the part after the prefix is the certificate's DNS SAN.

6.3 The request object — signed (ES256 JWS + x5c)

Served at `request_uri` as `application/oauth-authz-req+jwt` . Unlike an unsigned request, this is a **JWS**: `base64url(header)` . `base64url(payload)` . `base64url(signature)` (2628 bytes in the captured session).

JWS header — declares the signature algorithm and carries the Verifier certificate:

```
{ "x5c": [ "MIIBYjCCAXCgAwIBAgIJAK2bzgqFqB8x... (616-char DER certificate)" ],
  "typ": "oauth-authz-req+jwt",
  "alg": "ES256" }
```

JWS payload — the Authorization Request. Every field, its meaning, and the OpenID4VP 1.0 clause it comes from:

Field	Value (captured)	Meaning	OpenID4VP 1.0
<code>response_type</code>	<code>vp_token</code>	A verifiable presentation token is requested	section 5.2
<code>response_mode</code>	<code>direct_post.jwt</code>	The Holder returns the response by HTTP POST, encrypted as a JWE	section 8.3.1
<code>client_id</code>	<code>x509_san_dns:aadhaar-verifier</code>	The Verifier identity, prefixed with its client identifier scheme; trust the x5c cert whose SAN DNS matches the part after the prefix	section 5.9, section 5.9.3
<code>nonce</code>	<code>ea5edc81e3a9b90c709fbf7ce4624d59</code>	Per-session random value; binds the proof to this session	section 5.2
<code>response_uri</code>	<code>.../resp/9d7895aa...</code>	Where the Holder posts the response	section 8.2
<code>client_metadata</code>	(below)	The Verifier's metadata: encryption key and supported formats	section 5.1, section 11
<code>dcql_query</code>	(below)	The Digital Credentials Query	section 6

client_metadata — advertises the ephemeral key the Holder encrypts the response to, and the algorithms:

```
"client_metadata": {
  "jwks": { "keys": [ {
    "kty": "EC", "use": "enc", "crv": "P-256",
    "kid": "K4U39Gc1a5-cA04IsbWi3tLPfNjK8dNhg43FhSDNXDM",
    "x": "lZweR_wfbIeDb7sjmdiH-k6dXApoT7R5GSq9zJzctFQ",
    "y": "tKltx7M4sDX0tL53JDLBcpIq46EbL9zDNn3slrLghio",
    "alg": "ECDH-ES"
  } ] },
  "encrypted_response_enc_values_supported": [ "A256GCM" ],
  "vp_formats": { "mso_mdoc_zk": {} }
}
```

- jwks** (OpenID4VP 1.0 section 5.1) — a fresh, per-session EC P-256 public key generated by the Verifier; the Holder performs ECDH-ES to it. **kid** is its RFC 7638 JWK thumbprint, and **alg = ECDH-ES** names the key-agreement algorithm.
- encrypted_response_enc_values_supported** (OpenID4VP 1.0 section 5.1) — the supported JWE content-encryption (**enc**) algorithms; here **A256GCM** .
- vp_formats** — the accepted credential format, **mso_mdoc_zk** (the zero-knowledge mdoc format).

dcql_query (OpenID4VP 1.0 section 6) — the query:

```
"dcql_query": { "credentials": [ {
  "id": "aadhaar_zk",
  "format": "mso_mdoc_zk",
  "meta": {
    "doctype_value": "in.gov.uidai.aadhaar.1",
    "zk_system_type": [ { "system": "longfellow-libzk-v1",
      "id": "longfellow-libzk-v1_6_4_4283_2945_c70b5f44...49d6",
      "version": 6, "circuit_hash": "c70b5f44...49d6",
      "num_attributes": 4, "block_enc_hash": 4283, "block_enc_sig": 2945 } ]
  },
  "claims": [
    { "path": ["in.gov.uidai.aadhaar.1", "age_above18"] },
    { "path": ["in.gov.uidai.aadhaar.1", "age_above60"] },
    { "path": ["in.gov.uidai.aadhaar.1", "age_above50"] },
    { "path": ["in.gov.uidai.aadhaar.1", "age_above75"] }
  ]
} ] }
```


`credentials[].id` is a local handle the response is keyed by; `format = mso_mdoc_zk` marks the request zero-knowledge; `meta.doctype_value` is the requested document type; `meta.zk_system_type[]` advertises a circuit (a hint, section 5.3); each `claims[].path` is a `[namespace, element]` pair. `format`, `meta`, and `claims` are DCQL's `mso_mdoc` extensions (OpenID4VP 1.0 section 6.4); `zk_system_type` and `mso_mdoc_zk` are the Multipaz/longfellow zero-knowledge extension, not part of the standard (see section 15).

The complete decoded request is in [samples/request.decoded.json](#); the raw signed JWT is [samples/request.jwt](#). The Holder verifies and parses it (section 5.5, section 13.1); its log:

```
Fetching request_uri (get): https://oid4vp-relay.praveen-mdoc.workers.dev/req/9d7895aa...
Request signature verified and verifier certificate pinned OK
Parsed request: clientId=x509_san_dns:aadhaar-verifier zk=true docType=in.gov.uidai.aadhaar.1 claims=4 mode=direct_post.jwt
```

6.4 The response — encrypted (`direct_post.jwt` , JWE)

Because `response_mode = direct_post.jwt`, the Holder does not post the `vp_token` in the clear. It encrypts it to the Verifier's `client_metadata` key and posts the JWE (OpenID4VP 1.0 section 8.3, section 8.3.1):

```
response=<JWE compact serialization>[&state=<echoed if present>]
```

How the Holder builds it (`OpenId4VpZkClient.presentZk`):

```
val responseJson = JSONObject().put("vp_token", vpToken) // { "vp_token": { "aadhaar_zk": "<b64url DeviceResponse>" } }
val header = JWEHeader.Builder(JWEAlgorithm.ECDH_ES, EncryptionMethod.A256GCM)
    .keyID(encJwk.keyID).build() // kid = the verifier key advertised in client_metadata
val jwe = JWEObject(header, Payload(responseJson.toString()))
jwe.encrypt(ECDHEncrypter(encJwk)) // ephemeral-static ECDH-ES to the verifier key
formBuilder.add("response", jwe.serialize())
```

The plaintext that gets encrypted is the standard OpenID4VP response object (`{ "vp_token": ... }`, OpenID4VP 1.0 section 8.1); the `vp_token` value is exactly the object section 6-and-below would have posted in the clear.

The JWE, five-part compact form — `protected . encrypted_key . iv . ciphertext . tag`. Because `alg = ECDH-ES` is *direct* key agreement (not key-wrapping), the `encrypted_key` part is **empty**. Captured protected header (base64url-decoded):

```
{ "epk": { "kty": "EC", "crv": "P-256",
    "x": "Nr_sy0YVLz-829U5n3BGXQcuiJtR3v0QYTz-xzNEur8",
    "y": "U9RIVms9U1uq6EVI1Pjq6mWeDTix7raFUXJ-BLqRPM" },
  "kid": "K4U39Gc1a5-cA04IsbWi3tLPfNjK8dNhg43FhSDNXDM",
  "enc": "A256GCM",
  "alg": "ECDH-ES" }
```

- `epk` — the Holder's **ephemeral** public key; combined with the Verifier's private key it derives the content-encryption key (ECDH-ES).
- `kid = K4U39Gc1...` — **equals** the `kid` the request advertised in `client_metadata.jwks`, i.e. the Holder encrypted to exactly the key the Verifier asked for. (This same value is the RFC 7638 thumbprint folded into the transcript; section 10.1.)
- `enc = A256GCM`, `alg = ECDH-ES`.

Segment sizes in the captured JWE (`protected . encrypted_key . iv . ciphertext . tag`): protected header 291 chars, `encrypted_key` empty (direct ECDH-ES), `iv` 12 bytes, `ciphertext` the encrypted `{vp_token}` (649,720 chars ≈ 487 KB of proof), `tag` 16 bytes (AES-GCM authentication tag). The full JWE is in [samples/response.jwe.txt](#).

What the Verifier does with it (`0idVpVerifier.handleFormResponse`):

```
val jwe = params["response"]
val plaintext = VerifierCrypto.decryptResponse(jwe, encKey) // ECDHDecrypter with the session private key
val vpToken = JSONObject(plaintext).getJSONObject("vp_token") // { "aadhaar_zk": "<b64url DeviceResponse>" }
handleVpToken(vpToken) // → base64url-decode → DeviceResponse → verify
```

Decrypting yields the plaintext `{ "vp_token": { "aadhaar_zk": "<base64url DeviceResponse>" } }`; from there the flow is identical to an unencrypted response — the credential entry is base64url-decoded to the CBOR DeviceResponse and verified (section 13.4). Verifier log:

```
Decrypted direct_post.jwt response
Proof VERIFIED; issuer validated against UIDAI IACA (CN=UIDAI DS - in.gov.uidai.aadhaar.1, ...)
```


6.5 The relay contract

A Cloudflare Worker backed by a Durable Object (SQLite), one instance per session id. It stores and forwards two opaque objects and performs no verification (it never sees plaintext — the request is signed and the response is encrypted end-to-end between the two devices).

Method and path	Caller	Result
PUT /req/{id}	Verifier	signed request JWT stored → {"status":"stored"}
GET /req/{id}	Holder	the signed request JWT (application/oauth-authz-req+jwt), or 404
POST /resp/{id}	Holder	response=<JWE> stored → {"status":"ok"}
GET /resp/{id}	Verifier	the form body response=<JWE> (application/x-www-form-urlencoded), or 204

An empty GET /resp/{id} is held open for up to 25 seconds and returns the instant the Holder posts. Entries expire after 15 minutes. Any endpoint honouring this contract is compatible.

Captured round-trip (session 9d7895aa...):

```
GET .../req/9d7895aa... → 200 application/oauth-authz-req+jwt (2628 bytes; the signed request JWT)
POST .../resp/9d7895aa... → 200 {"status":"ok"}
GET .../resp/9d7895aa... → 200 application/x-www-form-urlencoded (response=<JWE>, 650062 bytes)
```

7. Transport B — Bluetooth Low Energy

The OpenID4VP request and the vp_token are the same as Transport A; the transport carries them over GATT with an encrypted channel, and uses the OpenID4VPBLEHandover transcript variant. The signed request (section 5.5) is carried inside the encrypted channel and verified + pinned by the Holder just as online. The Verifier is the GATT peripheral; the Holder is the GATT central. Code: ble/BleVerifierServer.kt , ble/BleWalletClient.kt , ble/BleProtocol.kt , ble/BleCrypto.kt .

7.1 The QR (engagement)

The Verifier displays:

```
OPENID4VP://connect?name=<url-encoded verifier name>&key=<hex X25519 public key, 64 hex characters>
```

- name** — the verifier's display name, URL-encoded.
- key** — the verifier's ephemeral X25519 public key (32 bytes as 64 lowercase hexadecimal characters), used to establish the encrypted channel.

7.2 GATT service and characteristics

Service 00000001-5026-444A-9E0E-D6F2450F3A77 . Characteristics share the suffix -5026-444A-9E0E-D6F2450F3A77 :

Short UUID	Name	Direction	Purpose
...0004	REQUEST_SIZE	Holder reads	Encrypted request size (4 bytes, big-endian)
...0005	REQUEST	Holder reads	Encrypted signed request, paged at ≤512 bytes per read
...0006	IDENTIFY	Holder writes	Holder X25519 public key (32 bytes) + nonce (12 bytes) = 44 bytes
...0007	CONTENT_SIZE	Holder writes	Encrypted vp_token size (4 bytes, big-endian)
...0008	SUBMIT_VC	Holder writes	Chunked encrypted vp_token
...0009	TRANSFER_SUMMARY_REQUEST	Holder writes	End-of-transfer sentinel (0x01)
...000A	TRANSFER_SUMMARY_REPORT	Verifier notifies	Acknowledgement
...000B	DISCONNECT	Verifier notifies	Session end

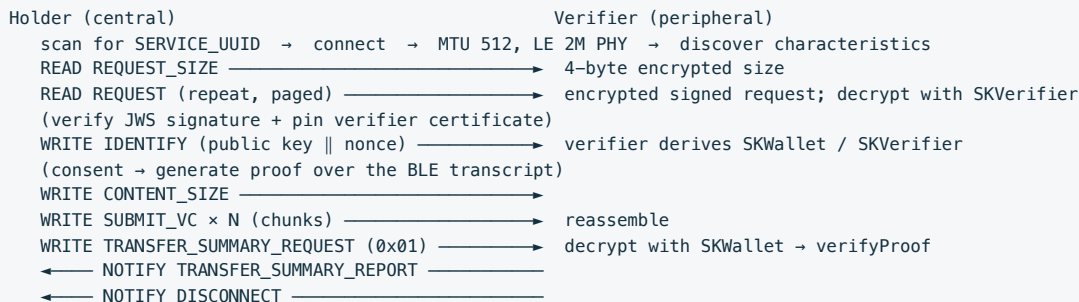
7.3 Encrypted channel

- Key agreement** — X25519 between the verifier's engagement key and the holder's ephemeral key, producing a 32-byte shared secret.

- **Key derivation** — HKDF-SHA256 (salt `null`) with two `info` strings: `"SKWallet"` (Holder → Verifier, the `vp_token`) and `"SKVerifier"` (Verifier → Holder, the signed request); each derives a 32-byte key.
- **Cipher** — AES-256-GCM; 12-byte IV = the holder's nonce from IDENTIFY; 16-byte tag; no additional authenticated data.
- **Framing** — each chunk is `seq (2 bytes, big-endian) || payload || CRC16-CCITT-FALSE (2 bytes, big-endian)` (polynomial `0x1021`, initial value `0xFFFF`); payload up to `min(MTU-3, 512) - 4`.
- **Radio** — the Holder requests MTU 512, high connection priority, and the LE 2M PHY.

The signed request itself is an ES256 JWS with `x5c` (larger than an unsigned request), so it spans several paged reads (see section 7.5). It is decrypted with `SKVerifier`, then verified and pinned by `VerifierRequestAuth` exactly as online (section 5.5) before consent.

7.4 Sequence



The decrypted request is the signed OpenID4VP request (`client_id = "mdoc-zk-verifier"` , a per-session `nonce` , the `x5c` certificate, and the same `dcql_query` structure as Transport A). The decrypted `vp_token` is the same DeviceResponse as Transport A.

7.5 Captured session

A Bluetooth presentation of four attributes with the signed request (Holder logs; complete log in [samples/ble-session.log](#)):

```

mtu=517
servicesDiscovered status=0
read REQUEST_SIZE len=4
read REQUEST len=509 (page 1)
read REQUEST len=509 (page 2)
read REQUEST len=509 (page 3)
read REQUEST len=229 (page 4)
Request signature verified and verifier certificate pinned OK
Selected circuit 'longfellow-libzk-v1_7_4_4415_4096_5aebda...e460' for 4 attribute(s)
ZK proof generated: 364516 bytes
submitting 365688 bytes in 720 chunks (mtu=517)
finish success=true
  
```

The MTU negotiated to 517 (effective characteristic write ≤ 512 bytes). The encrypted **signed** request (1756 bytes — larger than an unsigned request because it carries the `x5c` certificate and the ES256 signature) was paged across four reads. The Holder then **verified the signature and pinned the certificate** before generating the proof; the four-attribute circuit was selected; the encrypted `vp_token` (365,688 bytes) streamed as 720 chunks; `finish success=true`.

8. Transport C — NFC engagement to Bluetooth

NFC carries only the small engagement; the signed request and the proof are then transferred over Bluetooth (Transport B). The Verifier emulates an NFC Forum Type-4 tag (host-card emulation) under AID `F04D444F4356` and serves the `OPENID4VP://connect` engagement as a single NDEF URI record. The Holder is the NFC reader. Code: `nfc/NfcEngagementHceService.kt` (verifier), `NfcActivity.kt` / the reader path (holder).

8.1 Tag payload

A single NDEF URI record whose payload is the Bluetooth engagement string:

```

NDEF URI record: D1 01 <payload-len> 55 00 <URL bytes>
D1              record header (MB=1, ME=1, SR=1, TNF=well-known)
01              type length = 1
<payload-len>   length of (0x00 + URL)
55              type 'U' (URI)
00              URI abbreviation code = none
<URL bytes>     "OPENID4VP://connect?name=...&key=<64 hex X25519 public key>"

```

The Capability Container (file [E103](#)) and NDEF file ([E104](#)) are the standard Type-4 structures; the Capability Container content is:

```

00 0F  CCLen = 15
20      mapping version 2.0
00 3B  maximum read size (MLE)
00 34  maximum command size (MLC)
04 06  NDEF file control TLV (type 0x04, length 6)
E1 04  NDEF file id
00 FF  maximum NDEF size
00      read access: granted
FF      write access: denied

```

8.2 The tap: APDU exchange

The Holder enables reader mode, obtains the [IsoDep](#) channel, and transmits:

#	Reader → Tag (command APDU)	Tag → Reader (response)	Purpose
1	00 A4 04 00 06 F0 4D 44 4F 43 56	90 00	SELECT application by AID F04D444F4356
2	00 A4 00 0C 02 E1 04	90 00	SELECT the NDEF file (E104)
3	00 B0 00 00 02	LL MM 90 00	READ BINARY: the 2-byte NDEF length
4	00 B0 00 02 LL	D1 01 ... 90 00	READ BINARY: the NDEF message

The Holder parses the NDEF URI record, extracts the [OPENID4VP://connect...](#) URL, disables reader mode, and starts the Bluetooth central flow (Transport B, section 7.4) with that engagement — including the signed request verification.

8.3 Captured session

The Holder reads the engagement from the Verifier's tag over ISO-DEP, then continues over Bluetooth. The holder-side logs are identical to Bluetooth (NFC only replaces the QR scan). Complete log in [samples/nfc-session.log](#):

```

(NFC tap → read OPENID4VP://connect?name=Aadhaar+Verifier&key=<64 hex X25519 key>)
mtu=517
servicesDiscovered status=0
read REQUEST_SIZE len=4
read REQUEST len=509 x3 ; read REQUEST len=229      1756-byte signed request, paged
Request signature verified and verifier certificate pinned OK
Selected circuit 'longfellow-libzk-v1_7_4_4415_4096_5aebda...e460' for 4 attribute(s)
ZK proof generated: 364804 bytes
submitting 365976 bytes in 721 chunks (mtu=517)
finish success=true

```

The exchange proceeded exactly as Transport B: the same 1756-byte signed request was paged, verified, and pinned, then the four-attribute proof (365,976 bytes) streamed as 721 chunks. Because NFC hands the Bluetooth engagement off to the same BLE code, verifier authentication behaves identically on both offline transports.

9. Selective disclosure (ISO 18013-5)

This mode is native ISO 18013-5 proximity presentment to a standard mdoc reader; it is not zero-knowledge and not OpenID4VP. Code:

[Iso18013BleServer.kt](#), [Iso18013Crypto.kt](#).

- **Engagement** — a QR device engagement (*Show QR for reader*) or an NFC static handover (*Tap reader*); the reader then connects over Bluetooth.
- **Bluetooth service** — [0000ADB5-0000-1000-8000-00805F9B34FB](#), with State, Client2Server, and Server2Client characteristics (distinct from the zero-knowledge service in section 7.2).

- **Session security** — EC P-256 ECDH → HKDF-SHA256 → AES-GCM, with the IV per ISO 18013-5 section 9.1.1.5; session keys `SKDevice` , `SKReader` , `EMacKey` .
- **Reader request (DeviceRequest)**: `{ "version": "1.0", "docRequests": [ { "itemsRequest": 24(bstr( { "docType": "in.gov.uidai.aadhaar.1", "nameSpaces": { "in.gov.uidai.aadhaar.1": { "name": true, "address": true, ... } } } ) ) } ] }`
- **Holder response (DeviceResponse)** — the Holder filters its IssuerSigned items to the requested elements and returns `documents[]` with `issuerSigned` (the requested items plus `issuerAuth` = COSE_Sign1 over the MSO) and `deviceSigned` (a device MAC over the transcript). The reader verifies each value against its digest in the MSO, verifies the MSO signature, and verifies the device MAC. The actual values and the issuer signature are transmitted.

10. Session transcript

Both parties compute the transcript identically; the device signature (inside the proof, or the MAC for selective disclosure) is over it, so a proof is single-use and bound to the session and channel. The online transcript is the OpenID4VP 1.0 mdoc `OpenID4VPHandover` (Appendix B.2.6), as implemented by Multipaz.

10.1 Online — `OpenID4VPHandover`

```
SessionTranscript = [ null, null, [ "OpenID4VPHandover", SHA256(HandoverInfo) ] ]
HandoverInfo      = CBOR([ client_id, nonce, jwk_thumbprint, response_uri ])
```

The third element binds the response encryption:

- **Encrypted response (`direct_post.jwt` , this session)**: `jwk_thumbprint` is the RFC 7638 SHA-256 thumbprint of the Verifier's `client_metadata` encryption key — the same value as the JWE `kid` . Both sides fold it in, so a proof is bound to the exact encryption key used.
- **Unencrypted response (`direct_post`)**: the slot is `null` .

Captured value (session `9d7895aa...` ; complete bytes in [samples/session-transcript.hex](#)):

```
jwk_thumbprint      = 2b8537f467356b9f9c034e08b1b5a2ded94f7cd8caf1d361838dc58520cd5c33
                    (= base64url-decode("K4U39Gc1a5-cA04IsbWi3tLPfNjK8dNhg43FhSDNXDM"), the JWE kid)
SHA256(HandoverInfo) = 74b9724579a31b3c9bec739f17b8c264790c9b7bcb2db099e6f7794d36ffa4c9
SessionTranscript    = 83f6f682714f70656e494434565048616e646f766572
                    5820 74b9724579a31b3c9bec739f17b8c264790c9b7bcb2db099e6f7794d36ffa4c9
```

10.2 Bluetooth — `OpenID4VPBLEHandover`

```
SessionTranscript = [ null, null, [ "OpenID4VPBLEHandover", SHA256(digestInput) ] ]
digestInput       = client_id || nonce || verifierPublicKeyHex || walletPublicKeyHex
```

Binds the two X25519 keys in addition to `client_id` and `nonce` .

10.3 Selective disclosure — ISO 18013-5

```
SessionTranscript = [ DeviceEngagementBytes, EReaderKeyBytes, Handover ]
```

11. Error conditions

Condition	Detected by	Behaviour
Request unsigned, or signature invalid, or <code>x5c</code> leaf not the pinned certificate	Holder, before consent	Refused; no data released ("Verifier sent an unsigned request..." / signature / pin error)
Attribute value > 64 bytes or item > 119 bytes	Holder, before proving	Excluded from the proof; disabled in consent
Requested attribute count has no matching circuit	Holder	Reported; no proof is generated
Verifier requests N attributes, Holder discloses M ($M < N$)	Holder / Verifier	The proof is valid for the M disclosed attributes; the Verifier reconciles (see section 14)
<code>direct_post.jwt</code> response fails to decrypt	Verifier	Rejected: "Failed to decrypt direct_post.jwt response"
Document Signer certificate does not chain to the trusted IACA	Verifier, before the proof	Rejected: "Issuer certificate not trusted: ..."
Invalid proof, wrong transcript, or tampering	Verifier	<code>verifyProof</code> throws; the result reports failure with the reason
Circuit id not bundled by the Verifier	Verifier	"No circuit for spec ""
DeviceResponse without <code>zkDocuments</code>	Verifier	"DeviceResponse has no zkDocuments"
Holder does not respond (online)	Verifier	Poll deadline of 180 seconds elapses
Bluetooth chunk fails CRC	Both	The frame is rejected; the transfer fails without partial acceptance

12. Field reference

Item	Value
Aadhaar document type	<code>in.gov.uidai.aadhaar.1</code>
Zero-knowledge format	<code>mso_mdoc_zk</code> (plain: <code>mso_mdoc</code>)
Client identifier	<code>client_id = "x509_san_dns:aadhaar-verifier"</code> (prefix folded into <code>client_id</code> , OpenID4VP 1.0 section 5.9.3)
Request object	signed ES256 JWS, <code>typ = oauth-authz-req+jwt</code> , Verifier cert in <code>x5c</code>
Response mode	<code>direct_post.jwt</code> (online, encrypted); AES-256-GCM channel (Bluetooth/NFC)
Response encryption	JWE <code>alg = ECDH-ES</code> , <code>enc = A256GCM</code> , to <code>client_metadata.jwks</code> key
DCQL claim path	<code>[namespace, element]</code> , e.g. <code>["in.gov.uidai.aadhaar.1", "age_above18"]</code>
Circuit advertised in the request	<code>meta.zk_system_type = [{ system, id, version, circuit_hash, num_attributes, ... }]</code> (a hint)
Circuit used (authoritative)	<code>DeviceResponse.zkDocuments[0].documentData.zkSystemId</code>
Response object (plaintext)	<code>vp_token = { "<credId>": base64url(DeviceResponse) }</code>
Proof location	<code>DeviceResponse.zkDocuments[0].proof</code>
Disclosed values	<code>DeviceResponse.zkDocuments[0].documentData.issuerSigned</code>
Online transcript	<code>[null, null, ["OpenID4VPHandover", SHA256(CBOR([client_id, nonce, jwk_thumbprint null, response_uri]))]]</code>
Bluetooth transcript	<code>[null, null, ["OpenID4VPBLEHandover", SHA256(client_id nonce verifierHex walletHex)]]</code>
Size limits	value $\leq$ 64 bytes; IssuerSignedItem $\leq$ 119 bytes
Proof size (captured)	364,228 bytes ($\approx$ 480 KB base64)
Device key	<code>mdoc_device_key</code> , EC P-256, Android Keystore, ES256
Verifier signing/pinned certificate	EC P-256, self-signed, <code>CN = aadhaar-verifier</code> , SAN <code>DNS:aadhaar-verifier</code>
Zero-knowledge Bluetooth service	<code>00000001-5026-444A-9E0E-D6F2450F3A77</code>
Selective-disclosure Bluetooth service	<code>0000ADB5-0000-1000-8000-00805F9B34FB</code>
NFC engagement AID	<code>F04D444F4356</code>
Relay base URL	<code>https://oid4vp-relay.praveen-mdoc.workers.dev</code>

13. Worked example — the captured presentation, step by step

This section follows session `9d7895aa11e55fff042cc1723f5bd78b` from the signed request through verification, quoting the code that runs on each side. Every value is from the captured session.

13.1 The request, verified and decoded

The Verifier served the signed JWT in `samples/request.jwt` (2628 bytes) at `request_uri` . The Holder first **authenticates** it (section 5.5): parse the JWS, verify the ES256 signature against the `x5c` leaf's public key, and require that leaf to be byte-identical to the bundled `verifier_cert` . In the capture the leaf certificate is:

```
subject : C=IN, O=Aadhaar Verifier, OU=ZKP Verifier, CN=aadhaar-verifier
SAN      : DNS:aadhaar-verifier          ← matches client_id (x509_san_dns)
key      : EC P-256; self-signed, ES256
```

Its public key is byte-identical to the app-bundled `verifier_cert` (pin satisfied), so the Holder logs `Request signature verified and verifier certificate pinned OK` and only then reads the payload.

The header `base64url-decodes` to `{"x5c":["MIIBYjCC..."],"typ":"oauth-authz-req-jwt","alg":"ES256"}`; the payload decodes to the request in section 6.3 — `response_mode = direct_post.jwt`, `client_id = x509_san_dns:aadhaar-verifier`, `nonce = ea5edc81...`, the `client_metadata` encryption key (`kid = K4U39Gc1...`), and the four-attribute `dcql_query`. Holder log:

```
Parsed request: clientId=x509_san_dns:aadhaar-verifier zk=true docType=in.gov.uidai.aadhaar.1 claims=4 mode=direct_post.jwt
```

13.2 Proving — what the Holder did, in order

Entry point `OpenId4VpZkClient.presentZk`; proof generation `ZkPresentationManager.generateZkDocument`.

Step 1 — compute the session transcript (section 10.1). Because the response is encrypted, the third slot is the Verifier encryption key's RFC 7638 thumbprint (not `null`):

```
val readerThumbprint = encJwk.computeThumbprint().decode() // = base64url-decode(kid), 32 bytes
val handoverInfo = Cbor.encode(buildCborArray {
  add(clientId); add(nonce); add(Bstr(readerThumbprint)); add(responseUri)
})
val digest = Crypto.digest(Algorithm.SHA256, handoverInfo)
sessionTranscript = Cbor.encode(buildCborArray {
  add(Simple.NULL); add(Simple.NULL)
  add(buildCborArray { add("OpenID4VPHandover"); add(digest) })
})
```

Real value for this session:

```
jwk_thumbprint      = 2b8537f467356b9f9c034e08b1b5a2ded94f7cd8caf1d361838dc58520cd5c33
SHA256(HandoverInfo) = 74b9724579a31b3c9bec739f17b8c264790c9b7bcb2db099e6f7794d36ffa4c9
SessionTranscript    = 83f6f682714f70656e494434565048616e646f7665725820 74b97245...a4c9
```

Step 2 — reject oversized attributes (`checkSizes`): each of the four age flags is a short string and passes the 64-byte value / 119-byte item limits.

Step 3 — choose the circuit by attribute count. Four attributes → the four-attribute circuit; its id becomes the proof's `zkSystemId`:

```
val spec = zkSystem.getMatchingSystemSpec(zkSystem.systemSpecs, requestedClaims) // 4 attributes
// Holder log: Selected circuit 'longfellow-libzk-v1_7_4_4415_4096_5aebda...e460' for 4 attribute(s)
```

The selected circuit (version 7) differs from the one advertised in the request (version 6); the proof carries the version that was used (section 5.3).

Step 4 — filter the credential to the four requested attributes.

Step 5 — sign for holder binding. An ES256 COSE_Sign1 device signature is constructed over `DeviceAuthentication = ["DeviceAuthentication", SessionTranscript, docType, 24(bstr({}))]` with the device key; the circuit verifies it against the device key in the MSO.

Step 6 — generate the proof (native, ~1–2 s):

```
val zkDocument = zkSystem.generateProof(spec, document, Cbor.decode(sessionTranscript), timestamp)
// Holder log: ZK proof generated: 364228 bytes
```

Step 7 — wrap, encrypt, and post. The `DeviceResponse` is `base64url-encoded` into the `vp_token`, the response object is **encrypted** to the Verifier's key as a `direct_post.jwt` JWE (section 6.4), and posted:

```
val deviceResponse = ZkPresentationManager.buildZkDeviceResponse(zkDocument)
val vpToken = JSONObject().put(req.credentialId, base64url(deviceResponse)) // { "aadhaar_zk": "<b64url>" }
val responseJson = JSONObject().put("vp_token", vpToken)
val jwe = JWEObject(JWEHeader.Builder(ECDH_ES, A256GCM).keyID(encJwk.keyID).build(),
  Payload(responseJson.toString()))
jwe.encrypt(ECDHEncrypter(encJwk))
// POST response=<jwe> → Holder log: POSTing encrypted direct_post.jwt response; Verifier responded HTTP 200
```

13.3 The response, decrypted and decoded

The Verifier receives `response=<JWE>`, decrypts it with its session private key, and reads the `vp_token` (section 6.4). The decrypted plaintext is:


```
{ "vp_token": { "aadhaar_zk": "o2d2ZXJzaW9uYzEuMGt6a0RvY3VtZW50c4Gi ... (base64url DeviceResponse)" } }
```

The complete, untruncated vp_token is in [samples/response.vp_token.txt](#) and its full decode in [samples/response.decoded.txt](#).

Base64url-decoding the credential entry yields the CBOR `DeviceResponse`, which decodes in full to (CBOR diagnostic notation; `h'...` a byte string, `24(<< ... >>)` a byte string containing encoded CBOR, `0("...")` a tag-0 date-time; the 364,228-byte `proof` shown by length and prefix):

```
DeviceResponse = {
  "version": "1.0",
  "zkDocuments": [
    {
      "proof": h'bc9f0966466f32a2a95092491bc183475684f74a67e7fca3... (364228 bytes)',
      "documentData": 24(<< {
        "zkSystemId": "longfellow-libzk-v1_7_4_4415_4096_5aebdaafe17296a3ef3ca6c80c6e7505e09291897c39700410a365fb278e460",
        "docType": "in.gov.uidai.aadhaar.1",
        "timestamp": 0("2026-07-22T12:51:39Z"),
        "issuerSigned": {
          "in.gov.uidai.aadhaar.1": [
            { "elementIdentifier": "age_above18", "elementValue": "Yes" },
            { "elementIdentifier": "age_above60", "elementValue": "No" },
            { "elementIdentifier": "age_above50", "elementValue": "No" },
            { "elementIdentifier": "age_above75", "elementValue": "No" }
          ]
        },
        "deviceSigned": {},
        "msoX5chain": h'3082029230820237a0030201020211... (662 bytes, one DER X.509 certificate)'
      } >>)
    ]
  },
  "status": 0
}
```

Field by field:

Field	CBOR type	Value / meaning
<code>version</code>	tstr	<code>"1.0"</code> — ISO 18013-5 DeviceResponse version
<code>status</code>	uint	<code>0</code> — OK
<code>zkDocuments</code>	array	one <code>ZkDocument</code>
<code>zkDocuments[0].proof</code>	bstr	the longfellow proof; 364,228 bytes; prefix <code>bc9f0966...</code>
<code>documentData</code>	tag 24 (bstr of CBOR)	the disclosed data, encoded as a nested CBOR byte string
<code>...zkSystemId</code>	tstr	the circuit id the Verifier resolves (four-attribute, version 7)
<code>...docType</code>	tstr	<code>in.gov.uidai.aadhaar.1</code>
<code>...timestamp</code>	tag 0	<code>2026-07-22T12:51:39Z</code> — presentation time
<code>...issuerSigned</code>	map	namespace → array of disclosed items
<code>...issuerSigned[ns][n]</code>	map	exactly two keys: <code>elementIdentifier</code> and <code>elementValue</code>
<code>...deviceSigned</code>	map	empty; the device signature is proven inside the proof, not carried here
<code>...msoX5chain</code>	bstr	one DER X.509 certificate — the Document Signer certificate (section 5.4)

The Document Signer certificate in `msoX5chain` decodes to:

```
issuer   : C=IN, O=Unique Identification Authority of India, OU=IACA, CN=UIDAI IACA Root CA
subject  : C=IN, O=Unique Identification Authority of India, OU=Document Signer,
           CN=UIDAI DS - in.gov.uidai.aadhaar.1
key      : EC P-256; signature algorithm: ecdsa-with-SHA256
```

What each disclosed item does and does not contain. Each entry in `issuerSigned` carries only two fields: `elementIdentifier` (the attribute name, e.g. `age_above18`) and `elementValue` (the cleartext value, e.g. `"Yes"`). It does **not** carry the `digestID`, the `random` salt, or any attribute digest. A standard ISO 18013-5 `IssuerSignedItem` contains `digestID`, `random`, `elementIdentifier`, and `elementValue`; in the zero-knowledge document only the identifier and value are present. The value digests committed in the MSO are not transmitted — the circuit proves that the disclosed values are consistent with those committed digests without revealing the digests, the salts, or the MSO.

Summary of what is and is not transmitted. The response transmits the requested attribute identifiers and values, the circuit id, the timestamp, the Document Signer certificate, and the proof. It does not transmit the issuer's signature over the MSO, the MSO itself, the attribute digests or salts, the device signature, or any non-requested attribute.

13.4 Verification — what the Verifier did, in order

Entry point `OidVpVerifier.handleFormResponse` ; verification `ZkVerifier.verifyDeviceResponse` .

Step 1 — decrypt the response. Parse the form body, take `response` , and decrypt the JWE with the session encryption private key (the counterpart of the `client_metadata` key):

```
val plaintext = VerifierCrypto.decryptResponse(params["response"], encKey) // ECDH-ES / A256GCM
val vpToken   = JSONObject(plaintext).getJSONObject("vp_token")           // { "aadhaar_zk": "<b64url>" }
// Verifier log: Decrypted direct_post.jwt response
```

Step 2 — decode the DeviceResponse. Take the credential entry and base64url-decode it:

```
val deviceResponse = Base64.decode(vpToken.getString(vpToken.keys().next()), URL_SAFE or NO_PADDING or NO_WRAP)
ZkVerifier.verifyDeviceResponse(deviceResponse, transcript) // transcript computed independently (section 10.1)
```

Step 3 — extract the ZkDocument from the `zkDocuments` array.

Step 4 — read the circuit id and select the matching circuit. The id carried in the proof is looked up among the Verifier's bundled circuits — how both sides converge on the same circuit:

```
val specId = zkDoc.documentData.zkSystemSpecId // "longfellow-libzk-v1_7_4_4415_4096_5aebda...e460"
val spec    = zkSystem.systemSpecs.firstOrNull { it.id == specId }
?: return fail("No circuit for spec '$specId'")
```

Step 5 — validate issuer trust (`ZkVerifier.validateIssuer` , section 5.4). The Document Signer certificate from `msoX5chain` is validated against the bundled UIDAI IACA — within validity, signed by the IACA, issuer name matching. In this session it chained to `CN = UIDAI IACA Root CA` and passed.

Step 6 — verify the proof. One call performs the entire cryptographic check; it throws on any invalid proof:

```
zkSystem.verifyProof(zkDoc, spec, Cbor.decode(transcript))
```

It verifies, in zero-knowledge: a valid issuer signature exists over an MSO committing to the disclosed attributes (without the signature being present); the disclosed values are consistent with the MSO's digests; and the device signature over this exact transcript is valid against the device key in the MSO. Verifier log:

```
Proof VERIFIED; issuer validated against UIDAI IACA (CN=UIDAI DS - in.gov.uidai.aadhaar.1, ...)
```

Step 7 — build the result. No throw means success; the disclosed values are read for display, and the result screen shows "Issued by UIDAI":

```
Result(
  success = true,
  disclosedAttributes = { "age_above18":"Yes", "age_above60":"No", "age_above50":"No", "age_above75":"No" },
  zkSystemSpecId = "longfellow-libzk-v1_7_4_4415_4096_5aebda...e460",
  issuerCertSubject = "...CN=UIDAI DS - in.gov.uidai.aadhaar.1",
  holderBound = true
)
```

After successful verification the Verifier holds the four requested values, the identity of the issuer, and cryptographic assurance that the values are genuinely issuer-signed and bound to the resident's device — having received the request only from a Verifier the Holder authenticated, the response only in encrypted form, and neither the issuer's signature nor any other attribute.

14. Partial disclosure: fewer attributes disclosed than requested

The number of attributes named in the request and the number present in the proof are independent. The request states which attributes the Verifier would like; the number the proof actually contains is determined by the Holder at consent time. This follows from the circuit-selection mechanism

(section 5.3): the circuit is selected by the count of attributes the Holder discloses, and the Verifier resolves and verifies against the circuit named in the proof (`zkSystemId`), not the circuit advertised in the request.

The following describes a Verifier requesting four attributes while the Holder discloses three; the same reasoning applies to any case where the number requested exceeds the number disclosed.

1. **Proof generation.** `generateZkDocument` is invoked with the three disclosed attributes. `getMatchingSystemSpec` selects the three-attribute circuit. The proof's `zkSystemId` identifies that circuit, and `documentData.issuerSigned` contains exactly the three disclosed identifier/value pairs.
2. **Circuit resolution during verification.** `verifyProof` resolves the circuit from the proof's `zkSystemId` and verifies against the three-attribute circuit. Because the Verifier derives the circuit from the proof rather than from its own request, a three-attribute proof verifies correctly even though the request named four. (The captured session shows the same independence in another form: the request advertised circuit version 6 while the proof used version 7, and verification succeeded.)
3. **Scope of a successful verification.** A successful `verifyProof` establishes only that the disclosed values are genuine, issuer-signed, and bound to the device. It does not establish that any non-disclosed requested attribute was provided.
4. **Reconciliation by the Verifier.** Enforcing completeness is the Verifier's responsibility, at the application layer. After verification, the Verifier compares `Result.disclosedAttributes` against the set it requested, identifies any attribute not disclosed, and applies its policy: accept, reject, or re-request. Presenting the outcome in the interface — for example, "Verified: 3 of 4 requested; address not disclosed" — is the recommended production behaviour.

15. OpenID4VP 1.0 conformance map

This build targets **OpenID for Verifiable Presentations 1.0**. The table maps each construct to its clause and states conformance precisely. On the online transport every request/response parameter follows 1.0. The one construct outside any standard is the zero-knowledge credential format.

Construct (this build)	OpenID4VP 1.0 clause	Conformance
Authorization Request via <code>request_uri</code> (method <code>get</code>), JAR / RFC 9101	section 5, section 5.10	Conformant
Signed request object (JWS, ES256, <code>typ = oauth-authz-req+jwt</code> , <code>x5c</code>)	section 5 (Request Object)	Conformant
<code>x509_san_dns</code> client identifier — prefix folded into <code>client_id</code> (<code>client_id = "x509_san_dns:aadhaar-verifier"</code>)	section 5.9, section 5.9.3	Conformant
<code>response_type = vp_token</code> , <code>nonce</code>	section 5.2	Conformant
DCQL <code>dcql_query</code> , <code>credentials[]</code> , <code>claims[].path</code>	section 6, section 6.4	Conformant (mdoc claim paths)
<code>client_metadata.jwks</code> (per-session encryption key) + <code>encrypted_response_enc_values_supported</code>	section 5.1, section 11	Conformant
Encrypted response <code>direct_post.jwt</code> (JWE, <code>ECDH-ES</code> + <code>A256GCM</code>)	section 8.3, section 8.3.1	Conformant
Unencrypted response <code>direct_post</code> , <code>response_uri</code>	section 8.1, section 8.2	Conformant
<code>vp_token = { <credId>: base64url(DeviceResponse) }</code>	section 8.1, Appendix B.2	Conformant (mdoc encoding)
Session transcript <code>OpenID4VPHandover</code> with the encryption-key thumbprint	Appendix B.2, B.2.6	Conformant (as implemented by Multipaz)
<code>format = mso_mdoc_zk</code> , <code>meta.zk_system_type</code> , <code>zkDocuments</code> , longfellow proof	—	Not standardized. The zero-knowledge mdoc format is a Multipaz/longfellow extension; it is not defined by OpenID4VP or ISO 18013-5
Certificate pinning (leaf must byte-match a bundled cert)	— (extends section 5.9.3 trust)	Additive security. OpenID4VP leaves <i>how</i> the <code>x509_san_dns</code> certificate is trusted out of scope; pinning is a stricter local trust decision (a trust list of one)
Bluetooth / NFC transports for OpenID4VP messages	—	Custom transport. OpenID4VP is defined over HTTP / the Digital Credentials API; carrying its messages over BLE/NFC is bespoke to this setup

In one line. On the wire, the **online** flow follows OpenID4VP 1.0's high-assurance profile (signed `x509_san_dns` request via `request_uri`, DCQL, encrypted `direct_post.jwt` response with `encrypted_response_enc_values_supported`, `OpenID4VPHandover` transcript) — every request/response parameter is 1.0. The **zero-knowledge credential format** inside that envelope is a longfellow/Multipaz extension outside any standard; and **Bluetooth/NFC** are custom transports for the same messages. Certificate pinning is an additive local trust decision.

Spec reference. OpenID for Verifiable Presentations 1.0: https://openid.net/specs/openid-4-verifiable-presentations-1_0.html. Section numbers above are from the Final specification.